

QCM : Généralités .NET & C#

- .NET est Un framework de développement
- Quel composant permet l'exécution du code .NET ? : CLR
- Le code C# est d'abord transformé en : Code IL
- Quel est le rôle du JIT ? : Transformer IL en code natif
- ◆ **B. .NET Framework / .NET Core / .NET**
 - .NET Core est : Cross-platform
 - À partir de quelle version Microsoft a fusionné .NET Framework et .NET Core ? : .NET 5
 - .NET Standard est : Une norme de compatibilité
- ◆ **C. ASP.NET Core MVC**
 - Quel composant gère la logique métier ? : Contrôleur
 - Les fichiers Razor ont l'extension : .cshtml
 - Le symbole utilisé pour écrire du code C# dans Razor est : @
 - Où s'affichent les messages d'erreur de validation : la vue (asp-validation-for)
- ◆ **D. Routage & URLs**
 - Quelle URL appelle par défaut HomeController → Index ? : /Home
 - Dans /Home/Privacy, Privacy est : L'action
 - validation principale d'un formulaire : ViewModel
 - Que contient ModelState : l'état validation du modèle
- ◆ **E. Middleware & Pipeline**
 - Un middleware est : Un composant du pipeline http
 - Quel middleware permet l'utilisation des fichiers statiques ? : UseStaticFiles
 - Quel middleware restaure la session ? : UseSession
 - configure-t-on les middlewares : Program.cs
 - Le middleware de session stocke les données dans : HttpContext
 - ModelState.IsValid: Vérifier la validité des données du formulaire
- ◆ **F. Formulaires ASP.NET Core**
 - Quel objet permet de vérifier la validité d'un formulaire ? : ModelState
 - Pourquoi utiliser un ViewModel ? : Validation + simplification
 - Combien d'actions minimum pour gérer un formulaire MVC ? : 2
- ◆ **G. State Management (Session / Cookie / JWT)**
 - Quel type de données accepte la session par défaut ? : String et int
 - Quel package JSON est officiel depuis .NET Core 3 ? : JsonSerializer (System.Text.Json)
 - Une session est stockée : Côté serveur
 - créer cookie en net core : Response.Cookies.Append()
 - Comment lire un cookie ? : Request.Cookies[]
 - Par défaut, la session accepte : string et int
 - Pourquoi sérialiser un objet avant de le stocker en session ? : La session n'accepte pas les objets
- ◆ **H. Filtres ASP.NET Core**
 - Quel est l'objectif principal des filtres ? : Centralisation du code
 - Un filtre peut être appliqué sur : Un contrôleur ou une action
 - Quand s'exécute un filtre d'action : Avant et/ou après l'action
 - À quel sert OnActionExecuting : Vérifier ou bloquer l'accès avant l'action
- ◆ **I. ADO.NET Microsoft.Data.SqlClient**
 - ADO.NET est utilisé pour : Manipuler des bases de données
 - SqlCommand : Exécuter requête - transporte la requête SQL
 - ExecuteNonQuery est utilisé pour : INSERT / UPDATE / DELETE
 - ExecuteReader : SELECT – lit résultats ligne par ligne
 - Return : (SqlDataReader)
 - ExecuteScalar retourne : Une seule valeur
 - Pourquoi utiliser using avec SqlConnection ? : Libération automatique des ressources
 - ADO.NET est un ORM. ? : F - C'est une **API bas niveau**.

QCM 2////////GEN////////

- 1- Quel est l'avantage de RedirectToAction ? : Arrêt automatique de l'exécution
 - 2- RedirectToAction évite la resoumission du formulaire (PRG pattern)
- Quel type d'expiration est IdleTimeout ? : Sliding
 - Pourquoi utiliser les annotations plutôt que du code manuel ? : Validation centralisée
 - Pourquoi l'ordre des middlewares est important ? : Certains dépendent des précédents
 - Pourquoi éviter de laisser une connexion SQL ouverte ? : Saturation des connexions
 - Que se passe-t-il si on oublie Close() ou using ? : Risque de blocage des ressources
 - Pourquoi ExecuteScalar() est plus performant qu'un ExecuteReader() pour un COUNT ? : Une seule valeur retournée
 - **Différence AddSession / UseSession ?**
 - **AddSession** : ajoute le service de session
 - **UseSession** : active le middleware de session
 - Ordre du Pipeline :
 - UseHttpsRedirection
 - UseStaticFiles
 - UseRouting
 - UseSession
 - UseAuthorization
 - MapControllerRoute
 - Run
 - Contrôleur par défaut : **Home**
 - Action par défaut : **Index**
 - Bonnes pratiques MVC :
 - SQL hors contrôleur
 - contrôleur léger
 - ViewModel pour les formulaires
 - validation centralisée
 - C'est quoi la sérialisation ? : Transformer un objet en JSON
 - Pourquoi l'utiliser ? : Stocker un objet
 - Package officiel : System.Text.Json
 - Où on l'utilise souvent ? : Session

- Pourquoi utilise-t-on la sérialisation avec la session ? : Parce que la session n'accepte pas les objets (accepte **string / int**)
- Lequel est CORRECT pour stocker un objet en session ? : Session.SetString("u", JsonSerializer.Serialize(user));
- TODO (CRUD Session)**
 - Le CRUD TODO est : Un CRUD logique
 - Pourquoi utiliser RedirectToAction après POST : Respecter le PRG Pattern
 - Comment est généré l'id dans TODO : manuellement
- Pipeline & Divers**
 - AddSession() sert à : Ajouter le service de session
 - UseSession() sert à : Activer le middleware de session
 - L'ordre correct est : UseRouting → UseSession

RÉSUMÉ

MVC → Contrôleur logique
Session → serveur
TODO → CRUD logique
Category → CRUD ADO.NET
AuthFilter → sécurité
PRG → RedirectToAction

- Si on supprime RedirectToAction() après un POST, quel problème peut apparaître ? : La double soumission du formulaire (refresh = resubmit).
- Quelle est la différence fondamentale entre ViewBag et TempData ? : ViewBag dure **une requête**, TempData survit à une **redirection**.
- Pourquoi ExecuteScalar() est plus performant pour un COUNT(*) ? : Parce qu'il retourne **une seule valeur**, sans parcourir les lignes.
- Si UseSession() est placé avant UseRouting(), que se passe-t-il : session **ne fonctionne pas correctement** (ordre incorrect).
- Sans using, une connexion SQL se ferme automatiquement. ? : **Risque de fuite de connexions**.
- On peut stocker directement une List<T> dans la session. ? : **Il faut sérialiser en JSON**.

PRATIQUE////

- ✓ Créer un formulaire CRUD
- ✓ Enregistrer la catégorie dans la base de données
- ✓ Respecter l'architecture MVC
- ✓ Utiliser ADO.NET

Contrôleurs/

└─ CategoriesController.cs

Models/

└─ Category.cs

ViewModels/

└─ CategoryVM.cs

Repository/

└─ CategoryRepository.cs

Views/Categories/

└─ Index.cshtml

└─ Create.cshtml

└─ Edit.cshtml

Category.cs :

```
public class Category
{
    public int Id { get; set; }
    public string Libelle { get; set; }
}
```

CategoryVM.cs :

```
using System.ComponentModel.DataAnnotations;
public class CategoryVM
{
    public int Id { get; set; }

    [Required]
    [MinLength(3)]
    public string Libelle { get; set; }
}
```

CategoryRepository.cs :

```
using Microsoft.Data.SqlClient;
using System.Collections.Generic;
```

```
public class CategoryRepository
{
    private readonly string cs =
        "Data Source=.;Initial Catalog=MyDB;Integrated
        Security=True;Encrypt=False";
    // CREATE → INSERT
    public void Add(Category c)
    {
        using SqlConnection conn = new SqlConnection(cs);
        conn.Open();
        SqlCommand cmd = new SqlCommand(
            "INSERT INTO Category(Libelle) VALUES(@lib)", conn);
        cmd.Parameters.AddWithValue("@lib", c.Libelle);
        cmd.ExecuteNonQuery();
    }
}
```

```
// READ → SELECT
public List<Category> GetAll()
{
    List<Category> list = new();
    using SqlConnection conn = new SqlConnection(cs);
    conn.Open();
    SqlCommand cmd = new SqlCommand("SELECT * FROM
    Category", conn);
    SqlDataReader rd = cmd.ExecuteReader();
    while (rd.Read())
    {
        list.Add(new Category
        {
            Id = (int)rd["Id"],

```

```
            Libelle = rd["Libelle"].ToString()
        });
    }
    return list;
}
// UPDATE
public void Update(Category c)
{
    using SqlConnection conn = new SqlConnection(cs);
    conn.Open();
    SqlCommand cmd = new SqlCommand(
        "UPDATE Category SET Libelle=@lib WHERE Id=@id",
        conn);
    cmd.Parameters.AddWithValue("@lib", c.Libelle);
    cmd.Parameters.AddWithValue("@id", c.Id);
    cmd.ExecuteNonQuery();
}
// DELETE
public void Delete(int id)
{
    using SqlConnection conn = new SqlConnection(cs);
    conn.Open();
    SqlCommand cmd = new SqlCommand(
        "DELETE FROM Category WHERE Id=@id", conn);
    cmd.Parameters.AddWithValue("@id", id);
    cmd.ExecuteNonQuery();
}
// GET BY ID
public Category GetByld(int id)
{
    using SqlConnection conn = new SqlConnection(cs);
    conn.Open();
    SqlCommand cmd = new SqlCommand(
        "SELECT * FROM Category WHERE Id=@id", conn);
    cmd.Parameters.AddWithValue("@id", id);
    SqlDataReader rd = cmd.ExecuteReader();
    if (rd.Read())
    {
        return new Category
        {
            Id = (int)rd["Id"],
            Libelle = rd["Libelle"].ToString()
        };
    }
    return null;
}
CategoriesController.cs :
using Microsoft.AspNetCore.Mvc;
using System.Linq;

[AuthFilter]
public class CategoriesController : Controller
{
    private CategoryRepository repo = new();

    // READ
    public IActionResult Index()
    {
        return View(repo.GetAll());
    }
    // CREATE
    [HttpGet]
    public IActionResult Create()
    {
        return View();
    }
    [HttpPost]
    public IActionResult Create(CategoryVM vm)
    {
        if (!ModelState.IsValid)
            return View(vm);
        repo.Add(new Category { Libelle = vm.Libelle });
        return RedirectToAction("Index"); // PRG
    }
    // UPDATE
    [HttpGet]
    public IActionResult Edit(int id)
    {
        var c = repo.GetByld(id);
        return View(new CategoryVM { Id = c.Id, Libelle = c.Libelle
    });
    }
    [HttpPost]
    public IActionResult Edit(CategoryVM vm)
    {
        if (!ModelState.IsValid)
            return View(vm);
        repo.Update(new Category { Id = vm.Id, Libelle = vm.Libelle
    });
        return RedirectToAction("Index");
    }
    // DELETE
    public IActionResult Delete(int id)
    {
        repo.Delete(id);
        return RedirectToAction("Index");
    }
}
index.cshtml :
@model List<Category>
<a href="/Categories/Create">Ajouter</a>
<table>
@foreach (var c in Model)
{
```

```
<tr>
<td>@c.Libelle</td>
<td>
<a href="/Categories/Edit/@c.Id">Modifier</a>
<a href="/Categories/Delete/@c.Id">Supprimer</a>
</td>
</tr>
</table>
```

Create.cshtml & Edit.cshtml :

```
@model CategoryVM
<form method="post">
<input asp-for="Id" type="hidden" />
<label>Libelle</label>
<input asp-for="Libelle" />
<span asp-validation-for="Libelle"></span>
<button type="submit">Valider</button>
</form>
////L'accès au CRUD est réservé aux utilisateurs connectés////
```

AuthFilter.cs :

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.AspNetCore.Mvc;

public class AuthFilter : ActionFilterAttribute
{
    public override void
    OnActionExecuting(ActionExecutingContext context)
    {
        if (context.HttpContext.Session.GetString("login") == null)
        {
            context.Result = new RedirectToActionResult("Login",
            "User", null);
        }
    }
}
```

Ex2

Formulaire User / Register
 Nom – Prénom – Login –Email – tel –date de naissance –
 adresse- CIN – Password – Confirmation
 Controllers/
 └─ UserController.cs

ViewModels/
 └─ UserRegisterVM.cs

Views/User/
 └─ Register.cshtml
 UserRegisterVM.cs :

using System.ComponentModel.DataAnnotations;

```
public class UserRegisterVM
{
    [Required]
    [MinLength(3)]
    public string Nom { get; set; }
    [Required]
    [MinLength(3)]
    public string Prenom { get; set; }
    [Required]
    [MinLength(4)]
    public string Login { get; set; }
    [Required]
    [EmailAddress]
    public string Email { get; set; }
    [Required]
    [RegularExpression(@"^[A-Z][1,2][0-9][5,6]$",
    ErrorMessage = "CIN invalide")]
    public string CIN { get; set; }
    [Required]
    [DataType(DataType.Date)]
    public DateTime DateNaissance { get; set; }
    [Required]
    [Phone]
    public string Telephone { get; set; }
    [Required]
    public string Adresse { get; set; }
    [Required]
    [MinLength(6)]
    [DataType(DataType.Password)]
    public string Password { get; set; }
    [Required]
    [Compare("Password", ErrorMessage = "Les mots de passe ne
    correspondent pas")]
    [DataType(DataType.Password)]
    public string ConfirmPassword { get; set; }
}
```

UserController.cs :

using Microsoft.AspNetCore.Mvc;

```
public class UserController : Controller
{
    // GET : afficher le formulaire
    [HttpGet]
    public IActionResult Register()
    {
        return View();
    }
    // POST : récupérer le formulaire
    [HttpPost]
    public IActionResult Register(UserRegisterVM vm)
    {
        if (!ModelState.IsValid)
        {
            return View(vm);
        }
    }
}
```

```
}
// Ici : traitement (BD, session, etc.)
// Pour l'examen, ce n'est pas obligatoire
ViewBag.Message = "Inscription réussie";
return View(); }
```

TYPES GÉNÉRIQUES

Classe Tab<T> au lieu de Tab(int)

-Avantages

Réutilisabilité du code
 Sécurité de typage
 Souplesse

Pourquoi le mappage ?

Respect du principe Single Responsibility

AUTHENTIFICATION --LOGIN ET INSCRIPTION (MVC + SESSION)

• /ViewModels/LoginVM.cs

using System.ComponentModel.DataAnnotations;

```
public class LoginVM
{
    [Required]
    public string Login { get; set; }

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; }
}
```

• /ViewModels/RegisterVM.cs

using System.ComponentModel.DataAnnotations;

```
public class RegisterVM
{
    [Required] public string Nom { get; set; }
    [Required] public string Prenom { get; set; }
    [Required] public string Login { get; set; }

    [Required, MinLength(6)]
    [DataType(DataType.Password)]
    public string Password { get; set; }

    [Required, Compare("Password")]
    [DataType(DataType.Password)]
    public string ConfirmPassword { get; set; }
}
```

• /Controllers/UserController.cs

using Microsoft.AspNetCore.Mvc;
 using Microsoft.AspNetCore.Http;

```
public class UserController : Controller
{
    // ----- LOGIN -----
    [HttpGet]
    public IActionResult Login()
    {
        return View();
    }
    [HttpPost]
    public IActionResult Login(LoginVM vm)
    {
        if (!ModelState.IsValid)
            return View(vm);
        // Vérification simple (EXAMEN)
        if (vm.Login == "admin" && vm.Password ==
            "123")
        {
            HttpContext.Session.SetString("login",
            vm.Login);
            return RedirectToAction("Index", "Categories");
        }
        ModelState.AddModelError("", "Login ou mot de
        passe incorrect");
        return View(vm);
    }
    // ----- REGISTER -----
    [HttpGet]
    public IActionResult Register()
    {
        return View();
    }
    [HttpPost]
    public IActionResult Register(RegisterVM vm)
    {
        if (!ModelState.IsValid)
            return View(vm);
        // (EXAMEN) Pas de BD obligatoire ici
        // Après inscription → login automatique
        HttpContext.Session.SetString("login", vm.Login);
        return RedirectToAction("Index", "Categories");
    }
    // ----- LOGOUT -----
    public IActionResult Logout()
    {
        HttpContext.Session.Clear();
        return RedirectToAction("Login");
    }
}
```

Ex3 : TODO

Controllers/
 └─ TodoController.cs
 Models/
 └─ Todo.cs
 ViewModels/
 └─ TodoVM.cs
 Views/ToDo/
 └─ Index.cshtml
 └─ Create.cshtml

Todo.cs

```
public class Todo
{
    public int Id { get; set; }
    public string Titre { get; set; }
}
```

TodoVM.cs

using System.ComponentModel.DataAnnotations;

```
public class TodoVM
{
    [Required]
    [MinLength(3)]
    public string Titre { get; set; }
}
```

TodoController.cs

using Microsoft.AspNetCore.Mvc;
 using Microsoft.AspNetCore.Http;
 using System.Text.Json;
 using System.Collections.Generic;
 using System.Linq;

```
[AuthFilter]
public class TodoController : Controller
{
```

// READ

```
public IActionResult Index()
{
    var todos = GetTodos();
    return View(todos);
}
```

// CREATE (GET)

```
[HttpGet]
public IActionResult Create()
{
    return View();
}
```

// CREATE (POST)

```
[HttpPost]
public IActionResult Create(TodoVM vm)
{
    if (!ModelState.IsValid)
        return View(vm);
    var todos = GetTodos();
    todos.Add(new Todo
    {
        Id = todos.Count + 1,
        Titre = vm.Titre
    });
    SaveTodos(todos);
    return RedirectToAction("Index"); // PRG
}
```

// DELETE

```
public IActionResult Delete(int id)
{
    var todos = GetTodos();
    var todo = todos.FirstOrDefault(t => t.Id == id);
    if (todo != null)
        todos.Remove(todo);
    SaveTodos(todos);
    return RedirectToAction("Index");
}
```

// ===== SESSION HELPERS =====

```
private List<Todo> GetTodos()
{
    var json = HttpContext.Session.GetString("todos");
    if (json == null)
        return new List<Todo>();
    return JsonSerializer.Deserialize<List<Todo>>(json);
}
private void SaveTodos(List<Todo> todos)
{
    HttpContext.Session.SetString(
    "todos",
    JsonSerializer.Serialize(todos)
    );
}
```

• SÉRIALISATION / DÉSÉRIALISATION (JSON)

-Using : using System.Text.Json;
 -Sérialisation (objet → JSON) :
 string json = JsonSerializer.Serialize(obj);
 -Désérialisation (JSON → objet) :
 var obj = JsonSerializer.Deserialize<Type>(json);
 -JSON + SESSION :
 HttpContext.Session.SetString(
 "user",
 JsonSerializer.Serialize(user)
);
 var user = JsonSerializer.Deserialize<User>(
 HttpContext.Session.GetString("user")
);

---- VERSION EXAMEN : -----

```
// Sérialisation
string json = JsonSerializer.Serialize(obj);
HttpContext.Session.SetString("data", json);
// Désérialisation
var obj = JsonSerializer.Deserialize<Type>(
    HttpContext.Session.GetString("data"));
```